DIAGNÓSTICO DE REDE: PING, TRACEROUTE, SS, DIG E MTR

"A rede está lenta" é o chamado mais comum e o mais inútil: pode ser o cabo, o DNS que demora dois segundos, 3% de perda num salto do meio — ou o disco do banco. ***O que separa cinco minutos de uma tarde perdida é ter um método****: perguntas em ordem fixa, cada uma isolando uma camada.*

080
GUIAS LINUX

A ESCADA: SEIS PERGUNTAS EM ORDEM FIXA

1	o link existe?	`ip -br link show` `sudo ethtool enp0s3`	Cabo, porta do switch, Wi-Fi não associado. **Nada acima disso importa.** `Duplex: Half` ou `100Mb/s` em placa gigabit = autonegociação ou par rompido.
2	tenho endereço e rota?	`ip -br addr show` `ip route get 1.1.1.1`	DHCP, perfil do gerenciador, máscara errada, rota ausente. Um IP `169.254.x.x` **é o DHCP que não respondeu**.
3	alcanço o gateway?	`ping -c3 "$(ip route | awk '/^default/{print $3}')"`	Camada 2 local — ARP, VLAN, filtro no switch. Confirme com `ip neigh`: `INCOMPLETE` no gateway fecha o caso.
4	alcanço a internet por IP?	`ping -c3 1.1.1.1` `mtr -rwc50 1.1.1.1`	Roteamento, NAT, firewall de saída, MTU. É onde entram o `traceroute` e o `mtr`.
5	resolvo nomes?	`dig +short exemplo.com` `resolvectl query exemplo.com`	Falhou aqui — e **só** aqui. IP funciona, nome não: é o `resolv.conf`, e nenhum reboot de roteador resolve.
6	a porta responde?	`ss -tulpn` `nc -zv exemplo.com 443`	Serviço parado, escutando no endereço errado, firewall de destino, TLS.

A regra que faz o método funcionar: não pule degraus e não volte. Cada teste que passa elimina **definitivamente** tudo o que está abaixo — quem começa pelo degrau 6 ("o site não abre") investiga TLS num problema de cabo.

LER A SAÍDA: LATÊNCIA, JITTER E A PERDA QUE NÃO É PERDA

kali@kali: ~

```
$ ping -c4 1.1.1.1
64 bytes from 1.1.1.1: icmp_seq=1 ttl=57 time=12.4 ms
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max/mdev = 12.1/12.9/13.8/0.6 ms
$ mtr -rwc50 1.1.1.1
HOST                 Loss%  Snt  Last   Avg  Best  Wrst StDev
1. 192.168.1.1        0.0%   50   1.2   1.4   1.0   3.1   0.3
2. 100.64.0.1         0.0%   50  11.8  12.5  11.2  22.4   1.8
3. core-rj.exemplo 18.0%   50  13.1  13.9  12.4  28.7   2.6
4. 1.1.1.1            0.0%   50  12.9  13.2  12.6  15.1   0.4
# perda num salto que SOME nos seguintes NÃO é perda: é rate-
# limit de ICMP. Perda real vai até a ÚLTIMA linha.
```

AS QUATRO LEITURAS DO PING

`time`	RTT ida e volta: LAN < 1 ms · mesmo país 5–30 ms · intercontinental 100–250 ms
`mdev`	o **jitter**. Alto com média baixa arruína VoIP sem afetar download nenhum — é o "a internet está boa mas a chamada trava"
`ttl`	pista de distância: sistemas partem de 64, 128 ou 255; `ttl=57` sugere 7 saltos
`loss`	**1–2% já degrada TCP**: cada perda dispara retransmissão e corta a janela

`ping` **mudo não prova host fora do ar** — bloquear ICMP é padrão em borda e em nuvem. E responder não prova aplicação de pé: é camada 3, e só.

DEGRAU 6: QUEM ESCUTA, QUEM RESPONDE E O BURACO NEGRO DE MTU

kali@kali: ~

```
$ ss -tulpn
tcp LISTEN 127.0.0.1:5432  users:(("postgres",pid=812))
tcp LISTEN 0.0.0.0:22      users:(("sshd",pid=744))
# 127.0.0.1 = SÓ loopback: nenhum remoto conecta, por mais
# liberado que o firewall esteja.
$ dig exemplo.com   # `ss -tin` dá rtt/cwnd/retrans
;; ->>HEADER<<- opcode: QUERY, status: NOERROR
exemplo.com.  280 IN A 93.184.215.14
;; Query time: 24 msec ;; SERVER: 127.0.0.53#53
$ diff <(dig +short site) <(dig @1.1.1.1 +short site)
# difere: cache velho ou interceptação. Igual: não é o DNS.
```

O QUE O DIG RESPONDE

`status`	`NOERROR` sem resposta = domínio existe, esse registro não · `NXDOMAIN` = não existe · `SERVFAIL` = servidor ou DNSSEC
`TTL`	o número caindo a cada consulta prova que veio do **cache**

ESTADOS QUE DENUNCIAM

`TIME-WAIT`	aos milhares: a aplicação abre **conexão nova por requisição**
`CLOSE-WAIT`	acumulando: **bug na aplicação**, que não chama `close()`. Nenhum ajuste de kernel conserta
`recusada`	chegou e ninguém escuta — **o host está vivo**
`esgotada`	nada respondeu: firewall descartando em silêncio ou rota quebrada

O buraco negro de MTU atravessa a escada inteira sem ser pego por degrau nenhum: o `ping` passa, o DNS resolve, o TCP abre — e a transferência trava quando os pacotes crescem (o SSH conecta e congela ao listar um diretório). Algum enlace aceita menos que 1500 bytes e o ICMP que avisaria está bloqueado. Teste: `ping -M do -s 1472 alvo` falha e `-s 1400` passa.

RESUMO RÁPIDO

- ✓ **O método é a escada:** link → endereço e rota → gateway → IP → nomes → porta. Não pule degraus e não volte.
- ✓ `ping` **mede camada 3 e só:** mudo não é host morto; respondendo não é aplicação no ar.
- ✓ **No** `mtr`**, perda que some depois é rate-limit de ICMP** — vale a última linha.
- ✓ `127.0.0.1:porta` **no** `ss` explica metade dos "roda mas não conecto"; `CLOSE-WAIT` acumulando é bug da aplicação, não da rede.
- ✓ **Recusada × esgotada:** recusada = host vivo sem quem escute; esgotada = firewall calado ou rota quebrada.

NO KALI

Diagnóstico e reconhecimento são separados pela autorização, não pela ferramenta. Os mesmos `ping`, `dig` e `nc -z` contra a própria infraestrutura são rotina e, contra faixas de terceiros em volume, são a etapa inicial de acesso não autorizado (Lei 12.737/2012). **Ping bloqueado não é host fora do ar — e isso define levantamento:** varredura ICMP acha só uma fração dos hosts vivos, porque servidor Windows e nuvem não respondem por padrão; é para isso que existe o `nmap -Pn`. `dig` **é o reconhecimento mais educado que existe:** `NS`, `MX` e `TXT` leem dados publicados de propósito, sem tocar o alvo — já a transferência de zona (`dig axfr`) entrega o domínio inteiro: contra a própria zona é auditoria, contra zona alheia é acesso indevido. **Meça o caminho antes de varrer:** sobre enlace com 5% de perda o `nmap` reporta como fechadas portas abertas — o SYN se perdeu e o silêncio virou "filtrada". **RHEL:** `dig` vem em `bind-utils`, `mtr` costuma faltar, e o `firewalld` ativo dá timeout num `nc -zv` externo com o serviço no ar.